



RANGKUMAN LENGKAP

ADMINISTRASI BASIS DATA

Materi 7 · Materi 8 · Materi 9

Strategi Backup Data | Transaction Log & Point-in-Time Recovery | Pemeliharaan Database

Penjelasan super lengkap agar mahasiswa paham & siap menjawab soal ujian



MATERI 7 – STRATEGI BACKUP DATA

MENGAPA BACKUP ITU WAJIB?

Bayangkan kamu sudah input data mahasiswa selama satu semester, lalu tiba-tiba server mati dan semua data hilang. Itulah alasan backup HARUS dilakukan. Backup bukan hanya soal keamanan siber, tapi tentang KETERSEDIAAN DATA (Availability).

 **ANALOGI: Backup = Asuransi Data. Tanpa backup, satu insiden bisa menghancurkan seluruh sistem yang sudah dibangun.**

Ancaman yang diatasi dengan Backup:

- Human Error – contoh: DELETE FROM mahasiswa; (tanpa WHERE, semua data terhapus!)
- Kerusakan Hardware – hardisk rusak, RAM terbakar, server mati mendadak
- Serangan Malware/Ransomware – data dikunci atau dihapus oleh hacker
- Crash Server – sistem tiba-tiba mati tanpa peringatan
- Kesalahan Aplikasi – bug yang menghapus atau merusak data secara tidak sengaja



RECOVERY MODEL – 3 PILIHAN UTAMA

Recovery Model adalah PENGATURAN pada SQL Server yang menentukan bagaimana setiap perubahan data dicatat dan seberapa jauh kita bisa memulihkan database jika terjadi masalah. Pilih Recovery Model yang tepat = menentukan seberapa aman data kita.



SIMPLE Recovery Model

Mode paling dasar. Transaction log (catatan perubahan) dibersihkan OTOMATIS setelah setiap checkpoint. File log tetap kecil, tapi kemampuan recovery sangat terbatas.

-  Kelebihan: Mudah dikelola, file log kecil, tidak perlu backup log rutin
-  Kekurangan: Hanya bisa restore sampai backup TERAKHIR. Data setelah backup terakhir = HILANG PERMANEN
-  Tidak bisa Log Backup dan tidak bisa Point-in-Time Recovery
-  Cocok untuk: Database development, latihan, testing, data sementara/tidak kritis

 **JANGAN gunakan SIMPLE untuk database produksi, keuangan, atau akademik yang penting!**



FULL Recovery Model

Mode paling lengkap dan aman. SETIAP transaksi dicatat dengan detail penuh di dalam file transaction log (.ldf). Tidak ada auto-truncate – log harus dibackup secara rutin agar tidak memenuhi disk.

-  Mendukung Full Backup, Differential Backup, dan Transaction Log Backup
-  Bisa Point-in-Time Recovery (kembalikan ke menit/detik tertentu)
-  Data loss sangat minimal
-  File log (.ldf) bisa sangat besar jika tidak dibackup rutin
-  Butuh manajemen log backup terjadwal
-  Cocok untuk: Sistem akademik, keuangan, produksi – semua database penting

 **Gunakan FULL Recovery Model untuk semua database production!**

3 BULK-LOGGED Recovery Model

Mode khusus yang mirip FULL, TAPI operasi besar (import data masif) hanya dicatat secara minimal untuk menghemat ruang log. Biasanya dipakai sementara saat proses ETL besar, lalu dikembalikan ke FULL.

- Operasi yang di-log minimal: BULK INSERT, SELECT INTO, INDEX REBUILD
-  Tidak selalu mendukung point-in-time recovery saat operasi bulk sedang berjalan
-  Cocok untuk: Proses ETL (Extract-Transform-Load), import data skala besar

TABEL PERBANDINGAN RECOVERY MODEL

Fitur	SIMPLE	FULL	BULK-LOGGED
Log Backup	 Tidak bisa	 Bisa	 Bisa
Point-in-Time Recovery	 Tidak bisa	 Bisa	 Terbatas
Log Auto Truncate	 Otomatis	 Tidak (manual)	 Tidak (manual)
Risiko Data Loss	 TINGGI	 RENDAH	 SEDANG
Ukuran File Log	Kecil (ringan)	Bisa sangat besar	Sedang
Cocok untuk	Dev/Testing	Production/Kritis	ETL/Import besar

JENIS-JENIS BACKUP

A. Full Backup

Full Backup adalah backup LENGKAP dari seluruh database: semua tabel, struktur, prosedur, data, dan sebagian transaction log. Ini adalah FONDASI dari semua strategi backup – tanpa Full Backup, jenis backup lain tidak bisa digunakan.

- **Ukuran:** Paling besar di antara semua jenis backup

- **Waktu proses:** Paling lama
- **Fungsi utama:** Menjadi DASAR untuk Differential Backup dan Log Backup
- **Frekuensi ideal:** Harian (sistem kritis) atau mingguan (sistem biasa)

💡 **ANALOGI: Full Backup = Memfoto seluruh isi rumah. Setiap sudut, setiap barang, semuanya ter-capture.**

B. Differential Backup

Differential Backup hanya membackup DATA YANG BERUBAH sejak Full Backup TERAKHIR (bukan sejak differential terakhir – ini perbedaan kunci!). Semakin lama sejak Full Backup, semakin besar ukuran Differential Backup.

- **Ukuran:** Lebih kecil dari Full Backup (tapi terus bertambah seiring waktu sejak Full)
- **Waktu proses:** Lebih cepat dari Full Backup
- **Syarat restore:** HARUS ada Full Backup sebagai dasarnya

💡 **ANALOGI: Differential = Memfoto hanya barang-barang BARU yang datang sejak foto lengkap terakhir. Semakin lama tidak foto ulang lengkap, semakin banyak barang baru yang difoto.**

C. Ilustrasi Alur Backup & Restore – WAJIB DIPAHAMI

Skenario backup dan apa yang terjadi saat crash:

```
Senin pagi    → Full Backup          [FB]
Selasa pagi   → Differential Backup  [DB2] = perubahan sejak FB
Rabu pagi     → Differential Backup  [DB3] = perubahan sejak FB (lebih besar
dari DB2)
Rabu jam 14   → SERVER CRASH! DATA HILANG!
```

Prosedur Restore yang BENAR:

1. Restore Full Backup [FB] → gunakan WITH NORECOVERY (belum selesai, masih bisa lanjut)
2. Restore Differential TERBARU [DB3] → gunakan WITH RECOVERY (selesai, database siap pakai)
3. Database kembali ke kondisi Rabu pagi sebelum crash!

⚠️ **INGAT: Pakai DB3 (terbaru), BUKAN DB2! Differential terbaru sudah mencakup semua perubahan sejak Full Backup.**

🔑 PERINTAH SQL – UBAH RECOVERY MODEL

```
-- Mengubah ke FULL Recovery Model (untuk production)
ALTER DATABASE NamaDatabase SET RECOVERY FULL;

-- Mengubah ke SIMPLE Recovery Model (untuk development)
ALTER DATABASE NamaDatabase SET RECOVERY SIMPLE;
```

RANGKUMAN POIN KUNCI MATERI 7

4. Backup wajib untuk mengatasi human error, kerusakan hardware, malware, dan crash
5. 3 Recovery Model: SIMPLE (basic), FULL (lengkap), BULK-LOGGED (khusus import besar)
6. SIMPLE = tidak bisa log backup & tidak bisa point-in-time recovery
7. FULL = mendukung semua jenis backup dan point-in-time recovery
8. Full Backup = backup lengkap, menjadi dasar semua backup lain
9. Differential Backup = perubahan SEJAK FULL BACKUP terakhir (bukan sejak differential terakhir)
10. Restore: Full dulu (WITH NORECOVERY) → lalu Differential terbaru (WITH RECOVERY)

MATERI 8 – TRANSACTION LOG BACKUP & POINT-IN-TIME RECOVERY

APA ITU TRANSACTION LOG?

Transaction Log adalah file dengan ekstensi .ldf yang mencatat SETIAP perubahan yang terjadi pada database. Ini adalah 'buku harian' atau 'CCTV rekaman' database – setiap aktivitas tersimpan dengan detail.

Setiap operasi berikut dicatat di log SEBELUM diterapkan ke data:

- INSERT – data baru dimasukkan ke tabel
- UPDATE – nilai data diubah
- DELETE – baris data dihapus
- CREATE – tabel/objek baru dibuat
- ALTER – struktur objek dimodifikasi
- DROP – objek dihapus

 **ANALOGI: Transaction Log = CCTV database yang merekam 24/7. Kita bisa 'memutar ulang rekaman' untuk melihat dan memulihkan apa yang terjadi pada waktu tertentu.**

MEKANISME WRITE-AHEAD LOGGING (WAL)

SQL Server menggunakan mekanisme Write-Ahead Logging (WAL). Prinsipnya: perubahan SELALU dicatat di log TERLEBIH DAHULU, baru kemudian diterapkan ke file data. Ini menjamin konsistensi database dalam segala kondisi.

Contoh alur eksekusi perintah:

```
DELETE FROM Mahasiswa WHERE NIM = '001';
```

11. SQL Server mencatat operasi DELETE ke transaction log terlebih dahulu
12. SQL Server memastikan catatan log sudah aman tersimpan di disk
13. Baru kemudian perubahan diterapkan ke file data (.mdf)

 **Manfaat WAL: Jika listrik mati tiba-tiba? SQL Server membaca log saat startup dan mengembalikan database ke kondisi konsisten. Tidak ada data korup!**

TRANSACTION LOG BACKUP – FUNGSI & SYARAT

Transaction Log Backup adalah backup yang menyimpan ISI transaction log sejak log backup terakhir. Berbeda dengan Differential (mengacu ke Full), Log Backup mengacu ke LOG BACKUP SEBELUMNYA – membentuk rantai yang berurutan.

Tiga fungsi utama Transaction Log Backup:

- **1. Menyimpan riwayat transaksi:** untuk keperluan point-in-time recovery
- **2. Truncate (bersihkan) log:** menghapus bagian log yang sudah dibackup sehingga file tidak membesar
- **3. Kontrol ukuran .ldf:** mencegah disk penuh akibat log yang tidak dibersihkan

⚠ BAHAYA tanpa Log Backup rutin: File .ldf membesar terus → Disk penuh → SQL Server berhenti menerima transaksi → SISTEM MATI TOTAL!

Syarat melakukan Log Backup:

- Recovery Model HARUS = FULL (tidak bisa pada SIMPLE)
- Sudah pernah dilakukan Full Backup sebelumnya

ILUSTRASI ALUR LOG BACKUP – SKENARIO NYATA

Contoh di sistem akademik yang berjalan sepanjang hari:

```
08:00 → Full Backup selesai
09:00 → Log Backup #1 (menyimpan transaksi 08:00 - 09:00)
10:00 → Log Backup #2 (menyimpan transaksi 09:00 - 10:00)
10:30 → BENCANA: staf salah ketik DELETE tanpa WHERE!
        Semua data mahasiswa semester ini TERHAPUS!
10:29 → Inilah waktu target yang ingin dikembalikan
```

Prosedur Point-in-Time Recovery:

14. Restore Full Backup (08:00) → WITH NORECOVERY
15. Restore Log Backup #1 (09:00) → WITH NORECOVERY
16. Restore Log Backup #2 (10:00) → WITH STOPAT = '10:29:00', WITH RECOVERY

✅ HASIL: Database kembali PERSIS ke kondisi jam 10:29:00, 1 menit sebelum perintah DELETE yang salah! Semua data selamat!

LOG SEQUENCE NUMBER (LSN) – NOMOR URUT TRANSAKSI

LSN adalah nomor unik yang diberikan SQL Server untuk SETIAP transaksi yang masuk ke log. Bayangkan seperti nomor halaman dalam buku diary – halaman harus berurutan, tidak boleh ada yang hilang.

Aturan kritis tentang LSN:

- Log backup harus diapply dalam urutan LSN yang BERURUTAN dan TIDAK TERPUTUS

- Jika satu file log backup hilang di tengah rantai → restore GAGAL sepenuhnya
- Tidak bisa 'melompati' file log yang hilang

⚠ **Error umum: 'The log in this backup set begins at LSN...' = Ada file log backup yang hilang atau terlewat. Cek apakah rantai log backup lengkap!**

🕒 POINT-IN-TIME RECOVERY – PENJELASAN LENGKAP

Point-in-Time Recovery (PITR) adalah kemampuan mengembalikan database ke kondisi pada waktu yang PERSIS ditentukan – bisa sampai level detik. Ini hanya bisa dilakukan dengan Transaction Log Backup.

Syarat PITR bisa dilakukan:

- Recovery Model = FULL
- Ada Full Backup yang valid sebagai titik awal
- Ada rantai Log Backup yang BERURUTAN (tidak ada yang terlewat atau hilang)

Kasus-kasus yang diselesaikan dengan PITR:

- **DELETE tanpa WHERE:** Seluruh tabel terhapus → kembalikan ke 1 detik sebelum perintah itu dijalankan
- **UPDATE massal yang salah:** Ribuan data ter-update dengan nilai keliru → kembalikan ke sebelum UPDATE
- **Data korup akibat bug aplikasi:** Kembalikan ke waktu sebelum bug aktif dan merusak data

🛑 PARAMETER STOPAT – CARA KERJA & ATURANNYA

STOPAT adalah parameter dalam perintah RESTORE LOG untuk menghentikan proses restore pada titik waktu tertentu. Ini adalah kunci dari Point-in-Time Recovery.

Aturan penggunaan STOPAT – WAJIB DIINGAT:

- STOPAT HANYA digunakan pada log backup TERAKHIR yang direstore
- Semua log sebelumnya direstore dengan NORECOVERY (tanpa STOPAT)
- Log terakhir menggunakan STOPAT = waktu target + WITH RECOVERY

Struktur perintah restore Point-in-Time:

```
-- Step 1: Restore Full Backup
RESTORE DATABASE NamaDB FROM DISK = 'full.bak'
WITH NORECOVERY;

-- Step 2: Restore Log Backup #1 (log pertama)
RESTORE LOG NamaDB FROM DISK = 'log1.bak'
WITH NORECOVERY;

-- Step 3: Restore Log Backup #2 (log terakhir, gunakan STOPAT)
```

```
RESTORE LOG NamaDB FROM DISK = 'log2.bak'  
WITH STOPAT = '2025-01-15 10:29:00',  
RECOVERY;
```

⚠ JANGAN taruh STOPAT di log tengah-tengah! Gunakan STOPAT hanya di log backup TERAKHIR yang sudah mencakup waktu target recovery.

TAIL-LOG BACKUP – PENYELAMAT DARURAT

Tail-Log Backup adalah backup bagian PALING AKHIR dari transaction log yang belum sempat dibackup – diambil SEBELUM proses restore dimulai. Ini adalah langkah pertama yang harus dilakukan ketika database bermasalah tapi masih bisa diakses.

Kapan dan mengapa Tail-Log Backup dilakukan?

- Database mengalami masalah tapi file log masih bisa diakses
- Ingin melakukan restore namun tidak mau kehilangan transaksi terakhir
- SELALU dilakukan SEBELUM memulai proses restore apapun

⚠ Tanpa Tail-Log Backup: Semua transaksi dari log backup terakhir sampai saat crash HILANG SELAMANYA – tidak bisa dipulihkan!

RANGKUMAN POIN KUNCI MATERI 8

17. Transaction Log (.ldf) mencatat SETIAP perubahan menggunakan Write-Ahead Logging
18. WAL = log ditulis dulu, baru data diubah → menjamin konsistensi saat crash
19. Log Backup hanya bisa jika Recovery Model = FULL dan sudah ada Full Backup
20. Tanpa Log Backup rutin → file .ldf membesar → disk penuh → sistem mati
21. LSN = nomor urut transaksi; log harus direstore BERURUTAN atau gagal
22. Point-in-Time Recovery = kembalikan database ke jam/menit/detik tertentu
23. STOPAT = gunakan HANYA di log backup TERAKHIR dengan waktu target
24. Tail-Log Backup = backup darurat sebelum restore, WAJIB dilakukan jika log masih bisa diakses

MATERI 9 – PEMELIHARAAN DATABASE (DATABASE MAINTENANCE)

KONSEP DASAR: PREVENTIVE MAINTENANCE

Database yang digunakan terus-menerus (OLTP – Online Transaction Processing) akan mengalami penurunan performa secara bertahap akibat aktivitas INSERT, UPDATE, DELETE yang terus berjalan. Database Maintenance adalah upaya PENCEGAHAN, bukan perbaikan saat rusak.

 **ANALOGI:** Database seperti mesin kendaraan. Perlu servis rutin (ganti oli, cek ban) agar tetap optimal, BUKAN hanya diperbaiki saat mogok di jalan.

Tujuan Database Maintenance:

- 25. Menjaga performa query tetap cepat dan optimal
- 26. Memastikan execution plan yang dipilih optimizer adalah yang terbaik
- 27. Menjaga integritas fisik dan logis data
- 28. Mencegah terjadinya corruption (kerusakan data)

3 Komponen Utama Database Maintenance:

- Index Maintenance – mengatasi fragmentasi index
- Update Statistik – menjaga akurasi query optimizer
- DBCC CHECKDB – pemeriksaan integritas seluruh database

KOMPONEN 1: INDEX & FRAGMENTASI

Apa itu Index?

Index adalah struktur data berbentuk B-Tree (Binary Search Tree) yang dibuat SQL Server untuk MEMPERCEPAT pencarian data. Mirip daftar isi buku – langsung lompat ke halaman yang tepat tanpa baca satu persatu.

- **Tanpa Index:** SQL Server melakukan TABLE SCAN = membaca SEMUA baris dari awal hingga akhir (sangat lambat untuk data besar!)
- **Dengan Index:** SQL Server melakukan INDEX SEEK = langsung menuju data yang dicari (sangat cepat!)

Apa itu Fragmentasi Index?

Fragmentasi adalah kondisi di mana halaman-halaman data index sudah TIDAK TERSUSUN RAPI secara fisik di disk. Ibarat buku yang halaman-halamannya acak – daftar isi masih ada, tapi untuk sampai ke halaman yang dituju harus loncat-loncat ke tempat yang berbeda.

Penyebab Fragmentasi:

- INSERT data secara acak ke dalam clustered index (data tidak masuk urut)
- UPDATE pada kolom yang menjadi index (nilai berubah, posisi di index bergeser)
- DELETE data dalam jumlah besar (meninggalkan 'lubang kosong' di halaman)
- Operasi SHRINK file yang dilakukan berulang

Dua Jenis Fragmentasi:

Jenis	Definisi	Penyebab Utama	Dampak
Internal Fragmentation	Banyak ruang KOSONG di dalam halaman (page)	DELETE atau UPDATE data	Pemborosan ruang penyimpanan
External Fragmentation	Urutan halaman tidak berurutan secara fisik di disk	INSERT acak, DELETE massal	Disk read meningkat, query lambat

Cara Mengukur Fragmentasi – WAJIB HAFAL:

```
SELECT
    OBJECT_NAME(object_id) AS NamaTabel,
    index_id,
    avg_fragmentation_in_percent,
    page_count
FROM sys.dm_db_index_physical_stats
    (DB_ID(), NULL, NULL, NULL, NULL)
ORDER BY avg_fragmentation_in_percent DESC;
```

Interpretasi Nilai Fragmentasi & Tindakan:

Nilai Fragmentasi	Status	Tindakan	Penjelasan
0% – 5%	✅ Optimal	Tidak perlu tindakan	Index masih dalam kondisi baik
5% – 30%	⚠️ Sedang	REORGANIZE	Fragmentasi mulai terasa, perlu reorganisasi
> 30%	🔴 Tinggi/Parah	REBUILD	Fragmentasi parah, perlu rebuild total

💡 **CATATAN:** Perhatikan juga nilai `page_count`! Jika halaman sangat sedikit (< 1000), fragmentasi tinggi tidak terlalu berpengaruh signifikan pada performa.

REORGANIZE vs REBUILD – Perbedaan Kritis:

Aspek	REORGANIZE	REBUILD
Cara kerja	Menyusun ulang halaman yang ada secara online	Membuat index BARU dari awal, mengganti yang lama
Penggunaan Resource	● Ringan – tidak membebani sistem	● Berat – butuh ruang disk dan CPU besar
Blocking/Locking	Online – tidak memblok user lain mengakses data	Bisa menyebabkan blocking (tergantung versi SQL Server)
Hasil Defragmentasi	Mengurangi fragmentasi SEBAGIAN	Menghilangkan fragmentasi TOTAL/100%
Update Statistik?	✗ Tidak otomatis – harus update manual	✓ Otomatis memperbarui statistik
Kapan digunakan	Fragmentasi 5% – 30%	Fragmentasi > 30%

Sintaks Perintah Index Maintenance:

```
-- REORGANIZE: untuk fragmentasi ringan-sedang (5-30%)
ALTER INDEX ALL ON NamaTabel REORGANIZE;

-- REBUILD: untuk fragmentasi parah (>30%)
ALTER INDEX ALL ON NamaTabel REBUILD;
```

KOMPONEN 2: STATISTIK DATA (QUERY OPTIMIZER)

Apa itu Statistik?

Statistik adalah METADATA (data tentang data) yang menyimpan informasi tentang distribusi nilai dalam kolom-kolom tabel. SQL Server menggunakan statistik ini untuk membuat EXECUTION PLAN – rencana terbaik mengambil data.

 **ANALOGI: Statistik = Peta Jalan untuk Query Optimizer. Peta yang akurat → rute tercepat. Peta yang usang → nyasar, buang waktu dan bahan bakar!**

Fungsi Statistik untuk Query Optimizer:

- Mengestimasi jumlah baris yang dikembalikan (cardinality estimation)
- Memilih jenis JOIN yang tepat: Nested Loop, Hash Join, atau Merge Join
- Menentukan apakah menggunakan index atau table scan
- Membuat execution plan paling efisien untuk setiap query

Mengapa Statistik Bisa Usang/Tidak Akurat?

- INSERT besar-besaran – data bertambah drastis mengubah distribusi
- UPDATE massal – banyak nilai berubah sekaligus
- DELETE massal – distribusi data berubah signifikan
- Perubahan distribusi data seiring waktu yang tidak diimbangi update statistik

Dampak Statistik yang Usang:

- **Index tidak digunakan:** Optimizer memilih table scan padahal ada index yang cocok → query lambat
- **Execution plan salah:** Optimizer memilih strategi JOIN yang tidak tepat
- **CPU meningkat:** Query butuh lebih banyak resource karena jalur yang dipilih tidak efisien
- **Logical Read bertambah:** Lebih banyak blok data yang harus dibaca dari memori/disk

Perintah Update Statistik:

```
-- Update statistik untuk satu tabel tertentu
UPDATE STATISTICS NamaTabel;

-- Update statistik SELURUH database sekaligus
EXEC sp_updatestats;
```

Auto Update Statistics:

SQL Server secara DEFAULT sudah mengaktifkan Auto Update Statistics. Statistik diperbarui otomatis ketika sekitar 20% data berubah. Namun untuk database besar, auto update saja tidak cukup.

```
-- Cek status Auto Update Statistics
SELECT name, is_auto_update_stats_on FROM sys.databases;

-- Aktifkan Auto Update Statistics
ALTER DATABASE NamaDatabase SET AUTO_UPDATE_STATISTICS ON;
```

✅ **Best Practice: Selain mengandalkan auto update, lakukan manual UPDATE STATISTICS setelah operasi import data besar atau ETL!**

🔍 KOMPONEN 3: DBCC CHECKDB (INTEGRITY CHECK)

Apa itu DBCC CHECKDB?

DBCC CHECKDB adalah perintah untuk memeriksa KONSISTENSI LOGIS dan FISIK seluruh database. Ini seperti 'medical check-up' untuk database – memastikan semua bagian sehat dan tidak ada kerusakan tersembunyi.

Apa yang Dideteksi DBCC CHECKDB?

- Page Corruption – halaman data yang rusak secara fisik di disk
- Allocation Error – kesalahan pada peta alokasi ruang disk database
- Index Inconsistency – index tidak sinkron/sesuai dengan data aktual
- Metadata Error – kesalahan pada informasi objek-objek database

Penyebab Corruption yang Memerlukan CHECKDB:

- Listrik mati mendadak saat proses penulisan data
- Kerusakan fisik disk (bad sector)

- Crash sistem operasi yang tidak terduga
- Bug pada storage driver atau firmware

⚠ **Corruption yang tidak dideteksi dini bisa MENYEBAR dan MELUAS. Ketika akhirnya terdeteksi, mungkin sudah terlambat untuk recovery!**

Sintaks dan Mode DBCC CHECKDB:

```
-- Pemeriksaan standar (semua pesan ditampilkan)
DBCC CHECKDB ('NamaDatabase');

-- Sembunyikan pesan informasi, tampilkan error saja
DBCC CHECKDB ('NamaDatabase') WITH NO_INFOMSGS;

-- Mode CEPAT: hanya cek fisik, skip cek logis (lebih ringan)
DBCC CHECKDB ('NamaDatabase') WITH PHYSICAL_ONLY;
```

Opsi Perbaikan (SELALU backup dulu sebelum repair!):

```
-- Perbaikan ringan: rebuild index yang rusak (AMAN, tidak ada data loss)
DBCC CHECKDB ('NamaDatabase', REPAIR_REBUILD);

-- OPSI TERAKHIR: perbaikan dengan kemungkinan data loss
DBCC CHECKDB ('NamaDatabase', REPAIR_ALLOW_DATA_LOSS);
```

⚠ **REPAIR_ALLOW_DATA_LOSS = pilihan TERAKHIR yang hanya dijalankan jika tidak ada cara lain. Bisa menyebabkan kehilangan data! Pastikan backup terbaru tersedia!**

🔗 HUBUNGAN KETIGA KOMPONEN MAINTENANCE

Komponen	Fokus Utama	Dampak Jika Diabaikan	Frekuensi Ideal
Index (REBUILD / REORGANIZE)	Struktur fisik penyimpanan index	Query makin lambat, I/O disk meningkat, user mengeluh lambat	Mingguan
Statistik (UPDATE STATISTICS)	Akurasi estimasi & execution plan	Execution plan salah, CPU tinggi, query tidak efisien	Mingguan / setelah ETL besar
DBCC CHECKDB	Integritas fisik & logis database	Corruption menyebar, kehilangan data, database tidak reliable	Mingguan / Bulanan

✅ **KUNCI SUKSES: Maintenance yang baik = merawat KETIGA komponen secara berkala dan terjadwal. Bukan menunggu ada masalah, tapi MENCEGAH sebelum terjadi!**

RANGKUMAN POIN KUNCI MATERI 9

- 29. Database Maintenance = tindakan PENCEGAHAN rutin, bukan perbaikan saat rusak
- 30. Fragmentasi Index terjadi akibat INSERT/UPDATE/DELETE yang mengacak struktur fisik
- 31. Ukur fragmentasi dengan sys.dm_db_index_physical_stats
- 32. 0-5% = aman | 5-30% = REORGANIZE | >30% = REBUILD
- 33. REORGANIZE = ringan, online, sebagian. REBUILD = total, bisa blocking, otomatis update statistik
- 34. Statistik = metadata distribusi data untuk Query Optimizer membuat execution plan
- 35. Statistik usang → execution plan salah → query lambat dan CPU tinggi
- 36. Update statistik: UPDATE STATISTICS NamaTabel atau EXEC sp_updatestats
- 37. DBCC CHECKDB = pemeriksaan integritas menyeluruh (fisik & logis)
- 38. REPAIR_ALLOW_DATA_LOSS = opsi TERAKHIR, HANYA setelah backup tersedia

PANDUAN KILAT MENGHADAPI SOAL UJIAN

KATA KUNCI SOAL & JAWABANNYA

Kalau Soal Menyebut...	Maka Jawabannya Tentang...
'Recovery Model untuk database produksi/kritis'	→ FULL Recovery Model
'Tidak bisa melakukan log backup'	→ Recovery Model yang digunakan adalah SIMPLE, harus diubah ke FULL
'Point-in-Time Recovery gagal'	→ Recovery Model bukan FULL, atau ada Log Backup yang hilang (LSN mismatch)
'DELETE tanpa WHERE, bagaimana recovery?'	→ Point-in-Time Recovery: Full Backup + Log Backup dengan STOPAT sebelum DELETE
'File .ldf terus membesar / disk penuh'	→ Log Backup tidak dilakukan rutin → log tidak pernah di-truncate
'Query makin lambat seiring waktu'	→ Fragmentasi Index (REORGANIZE atau REBUILD tergantung persentase)
'Execution plan tidak optimal / CPU tinggi'	→ Statistik usang → perlu UPDATE STATISTICS
'Page corruption / integrity error'	→ Gunakan DBCC CHECKDB untuk deteksi dan perbaikan
'Fragmentasi 8%'	→ Gunakan REORGANIZE (5-30% = reorganize)
'Fragmentasi 55%'	→ Gunakan REBUILD (>30% = rebuild)
'LSN mismatch error saat restore log'	→ Ada file log backup yang hilang atau tidak diapply berurutan
'Database masih bisa diakses sebelum restore'	→ Lakukan Tail-Log Backup TERLEBIH DAHULU sebelum restore!
'REPAIR_ALLOW_DATA_LOSS'	→ Opsi perbaikan TERAKHIR untuk DBCC CHECKDB, bisa menyebabkan data loss
'NORECOVERY vs RECOVERY'	→ NORECOVERY = masih bisa dilanjutkan, RECOVERY = database selesai/siap pakai

RUMUS RESTORE – WAJIB DIINGAT!

Skenario 1: Full + Differential Backup

- | | |
|---------------------------------------|-------------------|
| 1. RESTORE DATABASE dari Full Backup | → WITH NORECOVERY |
| 2. RESTORE DATABASE dari Differential | → WITH RECOVERY |

Skenario 2: Point-in-Time dengan Log Backup

1. RESTORE DATABASE dari Full Backup → WITH NORECOVERY
2. RESTORE LOG dari Log #1 → WITH NORECOVERY
3. RESTORE LOG dari Log #2 → WITH NORECOVERY
4. RESTORE LOG dari Log terakhir → WITH STOPAT='YYYY-MM-DD HH:MM:SS',
RECOVERY

💡 **KUNCI: NORECOVERY = 'belum selesai, masih ada yang mau diapply'.
RECOVERY = 'sudah selesai, database siap dipakai'.**

🔗 RINGKASAN SUPER CEPAT (CHEAT SHEET)

Topik	Poin Kritis
SIMPLE Recovery	Tidak ada log backup, tidak ada PITR, hanya Full + Differential
FULL Recovery	Semua backup tersedia, PITR bisa, log harus dibackup rutin
Full Backup	Backup lengkap, dasar semua backup, ukuran paling besar
Differential Backup	Perubahan SEJAK Full terakhir, bukan sejak differential terakhir!
Log Backup	Perubahan sejak log backup sebelumnya, membersihkan log (.ldf)
LSN	Nomorurut transaksi, log harusurut tidak boleh ada yang hilang
STOPAT	Hanya di log TERAKHIR, menghentikan restore di waktu tertentu
Tail-Log Backup	Backup DARURAT sebelum restore jika log masih bisa diakses
Fragmentasi < 5%	Aman, tidak perlu tindakan
Fragmentasi 5-30%	REORGANIZE (ringan, online)
Fragmentasi > 30%	REBUILD (berat, tapi total)
REBUILD vs REORGANIZE	REBUILD otomatis update statistik, REORGANIZE tidak
UPDATE STATISTICS	Perbarui metadata distribusi data untuk optimizer
DBCC CHECKDB	Cek integritas fisik dan logis seluruh database

💎 Semangat Belajar! 💎

Rangkuman Administrasi Basis Data – Materi 7, 8 & 9